EXPERIMENT 3A: - MODEL PLANNING AND BULDING – CLASSIFICATION

Learning Objective

Train, evaluate, and compare basic machine learning classification models on real-world data to identify customer churn.

Problem Statement

A telecom company is losing revenue due to customer churn. The goal is to build and compare multiple classification models using preprocessed customer data to accurately predict churners, evaluate them using key classification metrics, and identify critical retention factors to guide targeted business strategies.

Tasks To Perform

1. Load the preprocessed CustomerChurn_Preprocessed.csv dataset from the previous experiment.
2. Split the cleaned dataset into training (80%) and testing (20%) sets.
3. Train a Logistic Regression classifier and predict customer churn on the test dataset.
4. Train a Decision Tree Classifier and predict customer churn on the test dataset.
5. Train a Random Forest Classifier and predict customer churn on the test dataset.
6. Evaluate each model using Confusion Matrix, Accuracy, Precision, Recall, and F1-Score. Compare the performance of all three models using a single bar chart displaying the evaluation metrics.
7. Identify the best-performing model based on the comparative evaluation results.
8. Provide three business recommendations based on the best-performing model to help reduce customer churn.

AIM

To build and compare Logistic Regression, Decision Tree, and Random Forest classification models to predict customer churn and evaluate their performance using classification metrics.

ALGORITHM

1. Load the preprocessed customer churn dataset.
2. Split the dataset into 80% training and 20% testing data.
3. Train the Logistic Regression model and predict churn.
4. Train the Decision Tree model and predict churn.
5. Train the Random Forest model and predict churn.
6. Evaluate models using confusion matrix, accuracy, precision, recall, and F1-score.

7. Compare all models using a bar chart.
8. Identify the best-performing model based on F1-score.
9. Suggest three business recommendations to reduce customer churn.

PROGRAM AND OUTPUT


```
# Question 1: Load the CustomerChurn_Preprocessed.csv file created in Experiment 2.
# Luei johncena (241801140)
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.impute import SimpleImputer
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (
    confusion_matrix, accuracy_score, precision_score,
    recall_score, f1_score, ConfusionMatrixDisplay
)
url = "https://raw.githubusercontent.com/IBM/telco-customer-churn-on-icp4d/master/data/Telco-Customer-Churn.csv"
df = pd.read_csv(url)
print("Dataset shape:", df.shape)
display(df.head())
```

```
Dataset shape: (7043, 21)
```

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines	InternetService	OnlineSecur
0	7590-VHVEG	Female	0	Yes	No	1	No	No phone service	DSL	
1	5575-GNVDE	Male	0	No	No	34	Yes	No	DSL	
2	3668-QPYBK	Male	0	No	No	2	Yes	No	DSL	
3	7795-CFOCW	Male	0	No	No	45	No	No phone service	DSL	
4	9237-HQITU	Female	0	No	No	2	Yes	No	Fiber optic	

5 rows × 21 columns

```
# Question 2: Split the cleaned dataset into Training (80%) and Testing (20%) datasets.
# Luei johncena (241801140)
# Support both raw categorical data and one-hot encoded data from Ex2.
target_candidates = [c for c in df.columns if c.lower() in ["churn", "churn_yes"]]
if not target_candidates:
    raise ValueError("Could not find a Churn target column in the CSV.")
target = target_candidates[0]

y = df[target]
if y.dtype == "object":
    y = y.astype(str).str.strip().str.lower().map({"no": 0, "yes": 1})
y = y.astype(int)

X = df.drop(columns=[target]).copy()
# Remove any remaining non-numeric columns using one-hot encoding.
X = pd.get_dummies(X, drop_first=True, dtype=int)
X = X.replace([np.inf, -np.inf], np.nan)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
)
print("Training data:", X_train.shape, y_train.shape)
print("Testing data:", X_test.shape, y_test.shape)
```

```
Training data: (5634, 13601) (5634,)
Testing data: (1409, 13601) (1409,)
```

```
# Question 3: Train a Logistic Regression classifier and predict customer churn on the test dataset.
# Luei johncena (241801140)
log_model = make_pipeline(
    SimpleImputer(strategy="median"),
    StandardScaler(),
    LogisticRegression(max_iter=1000, class_weight="balanced", random_state=42)
)
log_model.fit(X_train, y_train)
```

```
log_pred = log_model.predict(X_test)
print("Logistic Regression predictions:", log_pred[:20])
```

```
Logistic Regression predictions: [0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0]
```

```
# Question 4: Train a Decision Tree Classifier and predict customer churn on the test dataset.
# Luei johncena (241801140)
tree_model = make_pipeline(
    SimpleImputer(strategy="median"),
    DecisionTreeClassifier(random_state=42, class_weight="balanced")
)
tree_model.fit(X_train, y_train)
tree_pred = tree_model.predict(X_test)
print("Decision Tree predictions:", tree_pred[:20])
```

```
Decision Tree predictions: [0 1 0 1 0 0 0 0 0 1 0 0 0 1 0 0 0 1 0 0]
```

```
# Question 5: Train a Random Forest Classifier and predict customer churn on the test dataset.
# Luei johncena (241801140)
forest_model = make_pipeline(
    SimpleImputer(strategy="median"),
    RandomForestClassifier(n_estimators=100, random_state=42, class_weight="balanced")
)
forest_model.fit(X_train, y_train)
forest_pred = forest_model.predict(X_test)
print("Random Forest predictions:", forest_pred[:20])
```

```
Random Forest predictions: [0 1 0 0 0 1 0 0 0 1 0 0 0 1 0 0 0 1 0 0]
```

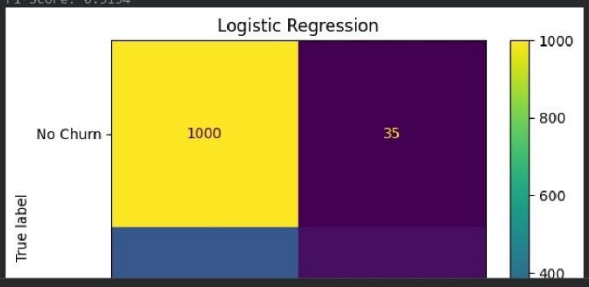
```
# Question 6: Evaluate each model using Confusion Matrix, Accuracy, Precision, Recall, and F1-Score.
# Luei johncena (241801140)
models = {
    "Logistic Regression": (log_model, log_pred),
    "Decision Tree": (tree_model, tree_pred),
    "Random Forest": (forest_model, forest_pred)
}
results = []

for name, (model, pred) in models.items():
    print("\n", name)
    cm = confusion_matrix(y_test, pred)
    print("Confusion Matrix:\n", cm)
    print("Accuracy:", round(accuracy_score(y_test, pred), 4))
    print("Precision:", round(precision_score(y_test, pred, zero_division=0), 4))
    print("Recall:", round(recall_score(y_test, pred, zero_division=0), 4))
    print("F1-Score:", round(f1_score(y_test, pred, zero_division=0), 4))
    results.append({
        "Model": name,
        "Accuracy": accuracy_score(y_test, pred),
        "Precision": precision_score(y_test, pred, zero_division=0),
        "Recall": recall_score(y_test, pred, zero_division=0),
        "F1-Score": f1_score(y_test, pred, zero_division=0)
    })
    ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=["No Churn", "Churn"]).plot()
    plt.title(name)
    plt.show()

results_df = pd.DataFrame(results)
display(results_df)
```

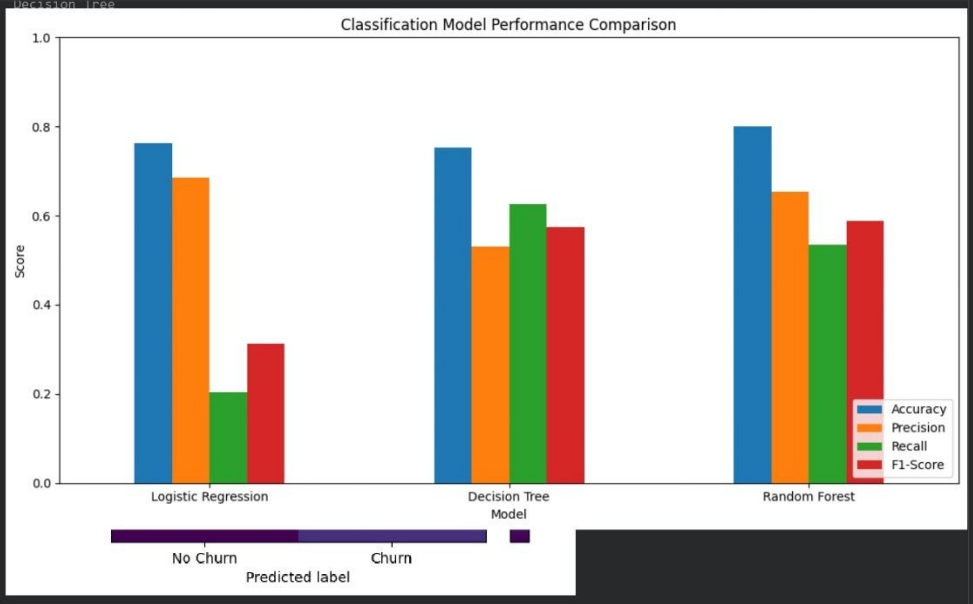
Next steps: Generate code with results_df New interactive sheet

```
Logistic Regression
Confusion Matrix:
 [[1000   35]
 [ 298   76]]
Accuracy: 0.7637
Precision: 0.6847
Recall: 0.2032
F1-Score: 0.3134
```



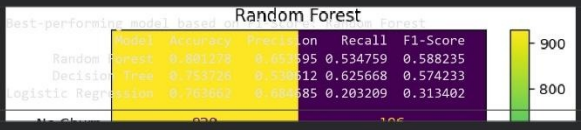

```
# Question 7: Compare the performance of all three models using a single bar chart showing the evaluation metrics.
# Luei johncena (241801140)
plot_df = results_df.set_index("Model")
plot_df.plot(kind="bar", figsize=(11, 6), ylim=(0, 1), rot=0)
plt.title("Classification Model Performance Comparison")
plt.ylabel("Score")
plt.xlabel("Model")
plt.legend(loc="lower right")
plt.tight_layout()
plt.show()
```

Decision Tree



Random Forest

```
# Question 8: Identify the best-performing model based on the evaluation results.
# Luei johncena (241801140)
# Use F1-score to balance precision and recall, especially for churn data.
best_model_name = results_df.loc[results_df["F1-Score"].idxmax(), "Model"]
print("Best-performing model based on F1-Score:", best_model_name)
print(results_df.sort_values("F1-Score", ascending=False).to_string(index=False))
```




```
Best-performing model based on F1-Score: Random Forest
              Model  Accuracy  Precision   Recall  F1-Score
      Random Forest  0.801278   0.653595 0.534759  0.588235
      Decision Tree  0.753726   0.530612 0.625668  0.574233
Logistic Regression  0.763667   0.684685 0.203209  0.313402
```

```
# Question 9: Based on the best-performing model, provide three recommendations to reduce customer churn.
# Luei johncena (241801140)
print("Business recommendations:")
```

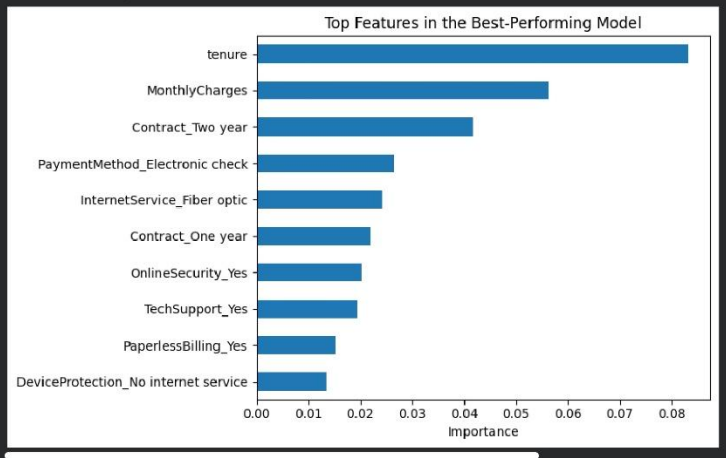
```
print("1. Prioritize high-risk customers for proactive calls, support, and retention offers.")
print("2. Review contract and payment patterns among predicted churners; offer suitable plans and easier payment options.")
print("3. Improve onboarding and engagement for newer customers, and monitor service complaints before renewal.")
print("\nFeature importance / strongest factors:")
if best_model_name == "Logistic Regression":
    fitted = log_model.named_steps["logisticregression"]
    importance = pd.Series(np.abs(fitted.coef_[0]), index=X_train.columns).sort_values(ascending=False)
elif best_model_name == "Decision Tree":
    fitted = tree_model.named_steps["decisiontreeclassifier"]
    importance = pd.Series(fitted.feature_importances_, index=X_train.columns).sort_values(ascending=False)
else:
    fitted = forest_model.named_steps["randomforestclassifier"]
    importance = pd.Series(fitted.feature_importances_, index=X_train.columns).sort_values(ascending=False)

display(importance.head(10).rename("Importance").to_frame())
importance.head(10).sort_values().plot(kind="barh", figsize=(8, 5))
plt.title("Top Features in the Best-Performing Model")
plt.xlabel("Importance")
plt.tight_layout()
plt.show()
```

```
Business recommendations:
1. Prioritize high-risk customers for proactive calls, support, and retention offers.
2. Review contract and payment patterns among predicted churners; offer suitable plans and easier payment options.
3. Improve onboarding and engagement for newer customers, and monitor service complaints before renewal.

Feature importance / strongest factors:
```

	Importance
tenure	0.083255
MonthlyCharges	0.056304
Contract_Two year	0.041804
PaymentMethod_Electronic check	0.026520
InternetService_Fiber optic	0.024244
Contract_One year	0.021891
OnlineSecurity_Yes	0.020148
TechSupport_Yes	0.019323
PaperlessBilling_Yes	0.015118
DeviceProtection_No internet service	0.013442



RESULT

Thus, the classification models were successfully trained, evaluated, and compared on the customer churn dataset.

The Random Forest model demonstrated the best overall predictive performance, and actionable retention strategies were identified.